ACADEMIC PROJECT PROGRESS & EVALUATION GUIDE

StayEase — 9-Week Academic Project Development Lifecycle

Complete Stage-by-Stage Documentation: Team Formation, SRS, UML Design, Database Schema, 4-Student Module Implementation & Integration Testing

METHODOLOGY	TEAM ALLOCATION	TEST PASS RATE	PROJECT OUTPUT
Agile-Iterative (9 Academic Weeks)	4 Students (1 Specialized Module Each)	295+ Automated Tests (100% Pass)	Production-Grade Web Application

1. Academic Lifecycle Timeline at a Glance

Week	Academic Stage	Key Engineering Deliverables	Status in StayEase
Week 1	Team Formation + Guide Selection + Abstract	Problem Identification, Guide Approval, Project Scope & Abstract	Completed Problem & Scope Defined
Week 2	Software Requirements Specification (SRS)	Functional & Non-Functional Requirements, User Personas, Constraints	Completed Full SRS Documented
Week 3	UML System Design	Use Case, Class, Sequence, Activity & Deployment Diagrams	Completed Complete Architectural Models
Week 4	Database Design & UI Mock-ups	3NF Relational Schema, ER Diagram, Wireframes & Thymeleaf Mockups	Completed MySQL/Postgres Schema + UI
Weeks 5–8	Module Coding (4 Individual Modules)	Student 1: Maps & Geocoding • Student 2: Booking Engine Student 3: Property Operations • Student 4: Security & Auth	Completed All 4 Modules Built & Documented
Week 9	System Integration & Testing	End-to-End Integration, 295+ Automated Tests, Docker & Cloud Deploy	Completed 100% Tests Pass + Cloud Live

2. Detailed Breakdown of Each Week: What Was Done & Deliverables

🗓️ WEEK 1: Team Formation, Guide Selection & Project Abstract

WEEK 1

Objective: Form a 4-member project team, consult faculty guide for topic approval, and submit the formal project abstract on the portal.

- **Problem Domain Identification:** Identified the critical housing and booking crisis faced by college students, interns, and young professionals who move to new cities. High broker fees (15% to 30%), unreliable availability on generic maps, and frequent double-bookings by manual hostel registers.
- **Project Abstract Formulated:** *"StayEase is a production-grade, zero-brokerage Hostel, PG, and Boutique Hotel Reservation Management Platform engineered with Java 21, Spring Boot 3, and PostgreSQL. It combines an editorial self-service booking portal with an open-source worldwide geospatial radar for guests, and an administrative console with hospitality financial analytics (ADR & RevPAR) and maintenance blackout controls for property owners."*
- **Team Roles Defined:** Divided into 4 distinct technical ownership domains across Architecture/Maps, Booking Concurrency, Operations/Analytics, and Security/Testing.
- **Guide Approval:** Scope approved covering web-based transactional booking, role-based authorization, and real-time open-source map discovery.

📋 WEEK 2: Software Requirements Specification (SRS)

WEEK 2

Objective: Elicit, document, and freeze all Functional (FR) and Non-Functional Requirements (NFR) for StayEase.

- **Functional Requirements (FRs):**
  1. **FR-1 (Auth & RBAC):** User registration, secure login, role separation between `ROLE_CUSTOMER` and `ROLE_ADMIN`.
  2. **FR-2 (Geospatial Discovery):** Search accommodations worldwide with 220ms debounced typeahead autocomplete and 10 km to 100 km radius filtering without Google API charges.
  3. **FR-3 (Transactional Booking):** Select dates, check live vacancy, calculate dynamic price, and confirm reservation with zero double-booking guarantee.
  4. **FR-4 (Receipts & Check-In):** Auto-generate formal digital booking vouchers with verification stamps and allow customer web check-in.
  5. **FR-5 (Operations & Maintenance):** Admin inventory CRUD, maintenance blackout dates blocking room availability, and seasonal price multiplier rules.
  6. **FR-6 (Audit Trail & Analytics):** Compute ADR, RevPAR, and occupancy rates, with immutable compliance audit logging and CSV export.
- **Non-Functional Requirements (NFRs):**
  - **Concurrency & Consistency:** Zero overbooking under simultaneous flash-booking traffic via ACID transactions.
  - **Security:** Salted password hashing (BCrypt), CSRF defense, HSTS, XSS mitigation, session fixation protection.
  - **Performance:** Sub-500ms response times for catalog searches; sub-2s for 100 km geospatial bounding box queries.
  - **Zero Operating Cost:** 100% open-source map ecosystem (OpenStreetMap, Leaflet, Photon) without API credit card dependencies.

📐 WEEK 3: UML System Design (Unified Modeling Language)

WEEK 3

Objective: Transform SRS into formal visual architectural diagrams to model static structure, dynamic behavior, and system flow.

- **1. Use Case Diagram:**
  - **Actors:** *Guest / Customer* (Browse rooms, use interactive map, book room, view bookings, download receipt, cancel stay, web check-in) and *Property Admin* (Manage rooms, approve/reject bookings, mark check-in/out, set maintenance blackouts, set seasonal pricing rules, export audit logs, view analytics).
- **2. Class Diagram (Entity-Relationship & Service Layer):**
  - Models entities: `User`, `Room`, `Booking`, `SeasonalPriceRule`, `RoomMaintenanceSchedule`, `AdminAuditLog`, and `InAppNotification`.
  - Defines Multiplicity: `User` 1:N `Booking`, `Room` 1:N `Booking`, `Room` 1:N `RoomMaintenanceSchedule`, `Room` 1:N `SeasonalPriceRule`.
- **3. Sequence Diagram (Booking with Concurrency Lock):**
  - `Customer` → `BookingController.bookRoom()` → `BookingService.createBooking()`.
  - `BookingService` triggers `RoomRepository.findByIdForUpdate(id)` (acquires `PESSIMISTIC_WRITE` row lock).
  - Queries `BookingRepository.findConflictingBookings(roomId, checkIn, checkOut)`.
  - If conflict exists → rollback and throw `BookingConflictException`; If clear → persist `Booking`, trigger `@Async` email/notification, commit transaction, and release lock.
- **4. State Machine / Activity Diagram:**
  - Booking Lifecycle: `PENDING` → `APPROVED` → `CHECKED_IN` → `CHECKED_OUT`. Alternate terminal states: `CANCELLED` and `NO_SHOW`.
- **5. Component & Deployment Diagram:**
  - Client Browser → Nginx Reverse Proxy / Port 8080 → Spring Boot 3 Embedded Tomcat → HikariCP Connection Pool → MySQL 8 / PostgreSQL → External Photon/OSM APIs.

🗄️ WEEK 4: Database Design & UI Mock-ups

WEEK 4

Objective: Design a normalized relational database schema (3NF) and build high-fidelity wireframes and UI templates.

- **Relational Database Schema (3NF Normalization):**

Table Name	Primary Key	Foreign Keys	Purpose & Constraints
`users`	`id` (BIGINT)	—	Stores login credentials, BCrypt password, unique email, role (`ROLE_CUSTOMER` , `ROLE_ADMIN`).
`rooms`	`id` (BIGINT)	—	Room number (unique), room type (`SINGLE` , `DOUBLE` , `TRIPLE` , `DORMITORY`), base price, active status.
`bookings`	`id` (BIGINT)	`user_id` , `room_id`	Check-in/out dates, status, total price, guest count, confirmation voucher code.
`room_maintenance_schedules`	`id` (BIGINT)	`room_id`	Start/end dates, reason, blocks room from search availability during maintenance.
`seasonal_price_rules`	`id` (BIGINT)	`room_id` (nullable)	Date range, multiplier/fixed price, applies festival/exam price surge automatically.
`admin_audit_logs`	`id` (BIGINT)	`admin_id`	Immutable audit trail: action performed, target entity, timestamp, details.

- **UI Mock-ups & View Prototypes (Thymeleaf & Bootstrap 5):**
  - **Public Landing & Map Wireframe:** Hero destination banner, search bar with typeahead suggestions, full-width Leaflet map with pin popups.
  - **Customer Dashboard Wireframe:** Upcoming stay spotlight card, past bookings history table with status badges, printable receipt modal.
  - **Admin Operations Console Wireframe:** Inventory KPI ribbon (Occupied, Vacant, ADR, RevPAR), tabbed booking manager, maintenance calendar modal.

3. Weeks 5–8: Module Coding (4 Students = 4 Specialized Modules)

In Weeks 5 to 8, development was divided among the 4 team members. Each student completely owned, developed, and unit-tested one major module:

MODULE 1 (STUDENT 1): Worldwide Geospatial Discovery & Open Map Engine

Code Packages: `com.stayease.service.places` , `com.stayease.controller.DiscoveryApiController` , `templates/discovery/index.html`

- **What Was Built:**
  1. **Leaflet.js Map Integration:** Interactive map tiles with custom colored pins (Blue for verified StayEase rooms, Orange for nearby OpenStreetMap stays).
  2. **Client-Side Debouncing:** Implemented 220ms typing debounce on the location input box to prevent high API call rates.
  3. **Komoot Photon Geocoding API:** Built fast typeahead autocomplete providing instant city/landmark suggestions with latitude and longitude.
  4. **Overpass & Nominatim Scanners:** Engineered dual-tier POI scanning: optimized regex Overpass queries for ≤35 km, switching to bounded viewbox Nominatim queries for 50–100 km radius, solving the public mirror timeout issue.
  5. **Haversine Formula:** Implemented spherical trigonometry in Java (`calculateDistanceKm`) to accurately compute surface distance from the user.

MODULE 2 (STUDENT 2): Customer Booking Engine & Concurrency Control

Code Packages: `com.stayease.service.BookingService` , `com.stayease.repository.RoomRepository` , `com.stayease.controller.BookingController`

- **What Was Built:**
  1. **Pessimistic Concurrency Locking:** Implemented `PESSIMISTIC_WRITE` row lock (`SELECT ... FOR UPDATE`) on rooms during reservation creation to eliminate double-booking race conditions during flash rushes.
  2. **Mathematical Date Overlap Algorithm:** Applied the canonical booking interval formula: `(existing.checkIn < new.checkOut) AND (existing.checkOut > new.checkIn)` .
  3. **Same-Day Back-to-Back Stays:** Strictly allowed same-day checkout (11:00 AM) and check-in (2:00 PM) transitions without false-positive conflict errors.
  4. **Dynamic Pricing Computation:** Integrated base price calculation with seasonal rules, guest multiplier, and stay duration.
  5. **Digital Printable Receipts:** Built automated guest vouchers (`/customer/bookings/{id}/receipt`) with verification stamps and customer web check-in.

MODULE 3 (STUDENT 3): Property Operations, Revenue Analytics & Compliance Audit

Code Packages: `com.stayease.service.MaintenanceScheduleService`, `com.stayease.service.SeasonalPricingService`, `com.stayease.controller.AdminOperationsController`

- **What Was Built:**
  1. **Room Maintenance Blackout Engine:** Allows admins to block specific rooms for repairs across custom date ranges, automatically hiding them from customer booking searches.
  2. **Dynamic Seasonal Pricing Engine:** Configured price multiplier rules for peak seasons (college fests, exams, holidays) with start/end dates.
  3. **Hospitality Financial Analytics:** Computed real-time **ADR (Average Daily Rate)** and **RevPAR (Revenue Per Available Room)** alongside occupancy rates.
  4. **Compliance Audit Trail:** Created immutable logging of administrative actions (approvals, rejections, check-ins, check-outs, no-shows) with CSV export.
  5. **Room Inventory Control:** Built full CRUD management for room inventory with active/inactive toggles and soft-delete safeguards.

MODULE 4 (STUDENT 4): Authentication, Spring Security, Notifications & Cron Reminders

Code Packages: `com.stayease.config.SecurityConfig`, `com.stayease.service.EmailService`, `com.stayease.service.BookingReminderService`

- **What Was Built:**
  1. **Spring Security 6 & RBAC:** Configured security filter chain separating `ROLE_CUSTOMER` (`/customer/**`) and `ROLE_ADMIN` (`/admin/**`).
  2. **BCrypt Password Hashing:** Applied adaptive salted password hashing with strength 10, neutralizing rainbow table attacks.
  3. **CSRF & Web Hardening:** Enforced synchronizer CSRF tokens across all POST forms, HSTS, X-Frame-Options: DENY, and session fixation protection.
  4. **Asynchronous Notification System:** Configured `ThreadPoolTaskExecutor` (`stayease-async-`, 5 core, 20 max threads) for non-blocking in-app alerts and emails.
  5. **Scheduled Cron Reminders:** Engineered `@Scheduled` cron job (`BookingReminderService`) auto-dispatching check-in reminder alerts one day prior to arrival.

🚀 WEEK 9: System Integration, Comprehensive Testing & Cloud Deployment

WEEK 9

Objective: Integrate all 4 modules, resolve interface boundaries, execute the automated test suite, containerize via Docker, and deploy live.

- **1. System Integration:**
  - Wired Security filters with Booking controllers and Admin operations.
  - Connected Leaflet frontend forms with Discovery REST API backend.
  - Unified flash message handlers, global exception handling (`@ControllerAdvice`), and database transaction rollbacks.
- **2. Comprehensive Automated Testing Suite (295+ Tests — 100% Pass Rate):**

Testing Level	Tools Used	What Was Tested & Verified	Result
Unit Testing	JUnit 5, AssertJ	Haversine formula calculations, price calculation logic, DTO validation.	100% Pass
Integration Testing	Spring Boot Test, H2	JPA repository CRUD, database constraints, entity relationships.	100% Pass
Concurrency Testing	Multithreaded Executor	Simultaneous booking requests on the same room; verified `PESSIMISTIC_WRITE` lock.	Zero Overbooking
Security & Web Tests	MockMvc, Spring Security Test	Role authorization (403 Forbidden on illegal URLs), CSRF token enforcement.	100% Pass

- **3. Docker Containerization & Cloud Deployment:**
  - Created multi-stage `Dockerfile` packaging Eclipse Temurin Java 21 JRE and compiled application JAR.
  - Configured `docker-compose.yml` orchestrating Spring Boot web app (port 8080) and MySQL 8 (port 3306).
  - Deployed live to Railway cloud with automated GitHub CI/CD synchronization.

4. Academic Evaluation Matrix (How to Defend This Timeline to the Professor)

Key Questions the Professor Can Ask About Your Timeline:

- **"How did you ensure all 4 students contributed equally?"**
  Answer: "Sir, in Weeks 5 to 8, we strictly partitioned the project into 4 independent modules: Student 1 owned Geospatial Discovery & Maps, Student 2 owned the Core Booking Engine & Concurrency, Student 3 owned Admin Operations & Financial Analytics, and Student 4 owned Spring Security & Async Notifications. Each student wrote their own services, controllers, entities, and unit tests."
- **"Why did you spend an entire week on UML (Week 3) and DB Design (Week 4)?"**
  Answer: "Because hotel reservation systems have complex concurrency and entity relations. Designing the Sequence Diagram for `SELECT ... FOR UPDATE` locking and 3NF database schema in advance prevented database redesigns and overbooking bugs during coding."
- **"How did you verify integration in Week 9?"**
  Answer: "We relied on our automated test suite of **295+ tests** built with JUnit 5 and MockMvc. Running `mvn clean test` verified that combining the security layer with the booking and admin modules caused zero regressions, maintaining a 100% pass rate."